Data Structures & Algorithms

A Concise Introduction

Chapter 1: Introduction

This textbook covers fundamental data structures and algorithms (DSA) that every computer science student must master. We begin with arrays and linked lists, progress through trees and graphs, and conclude with sorting and searching algorithms. Understanding these building blocks is essential for writing efficient software.

An algorithm is a step-by-step procedure for solving a problem. The efficiency of an algorithm is measured using Big-O notation (also called asymptotic notation), which describes how the running time or space requirements grow as the input size increases. Common complexities include O(1) constant time, O(log n) logarithmic, O(n) linear, O(n log n) linearithmic, and O(n^2) quadratic time.

Chapter 2: Arrays

An array is a contiguous block of memory that stores elements of the same type. Arrays provide O(1) random access by index but O(n) insertion and deletion in the worst case because elements must be shifted.

A dynamic array (also called a resizable array, ArrayList in Java, or vector in C++) automatically grows when capacity is exceeded, typically doubling its size. This gives amortized O(1) append operations.

Arrays are the foundation for many other data structures, including stacks, queues, and hash tables. Understanding arrays and their memory layout is a prerequisite for studying more complex structures.

Chapter 3: Linked Lists

A linked list (LL) is a linear data structure where each element (node) contains data and a pointer (reference) to the next node. Unlike arrays, linked lists do not require contiguous memory.

Types of linked lists:
- Singly Linked List (SLL): each node points to the next node only
- Doubly Linked List (DLL): each node points to both next and previous nodes
- Circular Linked List: the last node points back to the first

Linked lists support O(1) insertion and deletion at known positions but O(n) search since you must traverse from the head. They are used to implement stacks, queues, and adjacency lists for graphs.

Chapter 4: Stacks and Queues

A stack is a LIFO (Last In, First Out) data structure. The primary operations are push (add to top) and pop (remove from top), both O(1). Stacks can be implemented using arrays or linked lists.

Applications of stacks: function call management (call stack), expression evaluation, backtracking algorithms, undo operations in editors, and DFS (depth-first search).

A queue is a FIFO (First In, First Out) data structure. The primary operations are enqueue (add to rear) and dequeue (remove from front). A priority queue (PQ) is a variant where elements are dequeued by priority rather than insertion order.

Queues are used in BFS (breadth-first search), task scheduling, and buffering. A deque (double-ended queue) allows insertion and removal at both ends.

Chapter 5: Trees

A tree is a hierarchical data structure consisting of nodes connected by edges. The topmost node is the root. Each node can have zero or more children.

A Binary Tree (BT) is a tree where each node has at most two children (left and right). A Binary Search Tree (BST) is a binary tree with the property that for every node, all values in the left subtree are smaller and all values in the right subtree are larger.

BST operations: search, insert, and delete all run in O(h) where h is the height. For a balanced BST, h = O(log n). Unbalanced BSTs degenerate to O(n).

Self-balancing BSTs include AVL trees (strict balance, O(log n) guaranteed) and Red-Black trees (RB trees), which are used in many standard library implementations (e.g., TreeMap in Java, std::map in C++).

Chapter 6: Heaps and Priority Queues

A heap is a complete binary tree that satisfies the heap property. In a min-heap, every parent is smaller than its children; in a max-heap, every parent is larger.

Heaps are typically implemented using arrays. For a node at index i, its left child is at 2i+1 and right child at 2i+2. This array representation requires understanding of arrays as a prerequisite.

A binary heap supports insert and extract-min/max in O(log n) and find-min/max in O(1). Heaps are the standard implementation for priority queues (PQs).

Heap sort uses a heap to sort an array in O(n log n) time. Building a heap from an unsorted array (heapify) runs in O(n) time.

Chapter 7: Hash Tables

A hash table (also called hash map, dictionary, or associative array) stores key-value pairs. It uses a hash function to compute an index into an array of buckets.

Average-case complexity: O(1) for insert, search, and delete. Worst case: O(n) when many collisions occur.

Collision resolution strategies:
- Chaining: each bucket is a linked list of entries
- Open addressing: probe for the next empty slot (linear probing, quadratic probing, double hashing)

The load factor (number of entries / number of buckets) determines when to resize. Hash tables require arrays as the underlying storage and benefit from understanding of linked lists for chaining.

Chapter 8: Graphs

A graph G = (V, E) consists of vertices (nodes) V and edges E connecting pairs of vertices. Graphs can be directed or undirected, weighted or unweighted.

Graph representations:
- Adjacency matrix: 2D array, O(V^2) space, O(1) edge lookup
- Adjacency list: array of linked lists, O(V+E) space, preferred for sparse graphs

Graph traversal algorithms:
- BFS (Breadth-First Search): uses a queue, explores level by level, finds shortest path in unweighted graphs
- DFS (Depth-First Search): uses a stack (or recursion), explores as deep as possible before backtracking

Prerequisites: understanding of arrays, linked lists, stacks, and queues is essential for implementing graph algorithms.

Chapter 9: Sorting Algorithms

Sorting is the process of arranging elements in a specific order.

Comparison-based sorting algorithms:
- Bubble Sort: O(n^2), simple but inefficient
- Selection Sort: O(n^2), finds minimum repeatedly
- Insertion Sort: O(n^2) worst case, O(n) best case, good for small/nearly sorted data
- Merge Sort: O(n log n), stable, divide-and-conquer, requires O(n) extra space
- Quick Sort (QS): O(n log n) average, O(n^2) worst case, in-place, widely used
- Heap Sort: O(n log n), in-place, uses a heap data structure

Non-comparison sorts: Counting Sort, Radix Sort, Bucket Sort can achieve O(n) for restricted input domains.

Sorting requires understanding of arrays. Merge sort uses recursion (divide and conquer). Heap sort requires understanding of heaps. Quick sort uses partitioning.

Chapter 10: Searching Algorithms

Searching is the process of finding a target element in a data structure.

Linear Search: O(n), works on unsorted data, checks each element sequentially.

Binary Search (BS): O(log n), requires sorted data. Repeatedly divides the search space in half. Can be implemented iteratively or recursively. Binary search is a prerequisite concept for understanding BSTs.

Interpolation Search: improves on binary search for uniformly distributed data, achieving O(log log n) average case.

Hash-based search: O(1) average using hash tables.

Search in trees: BST search is O(log n) for balanced trees. For graphs, BFS and DFS are the primary search methods, each requiring different data structures (queue vs stack).